



Checkpoint III: First Prototype

Group: G16

Date: 2025/09/30

Prototype Architecture

The prototype is organized into **modular components** that separate responsibilities between layout, filtering, data handling, and visualization. This architecture ensures that the system is easy to extend as new idioms are implemented.

1. HTML Structure

The HTML file defines the overall dashboard layout:

- **Filters panel** (years, series, sets, types) on the left.
- **Charts section** on the right, organized into a top row (scatterplot and bar chart) and a bottom row (line chart with slider and ViolinPlot).
- Each visualization has its own dedicated container. This ensures consistency in layout and easy integration of additional idioms.

2. CSS Styling

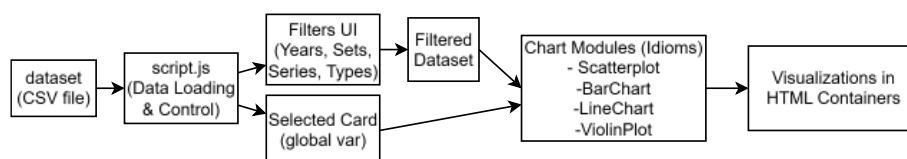
The style.css file manages the dashboard's look and feel. It ensures a cohesive design across idioms, maintaining consistent fonts, colors, and margins. This avoids the "patchwork" effect of disconnected visualizations.

3. JavaScript Modules

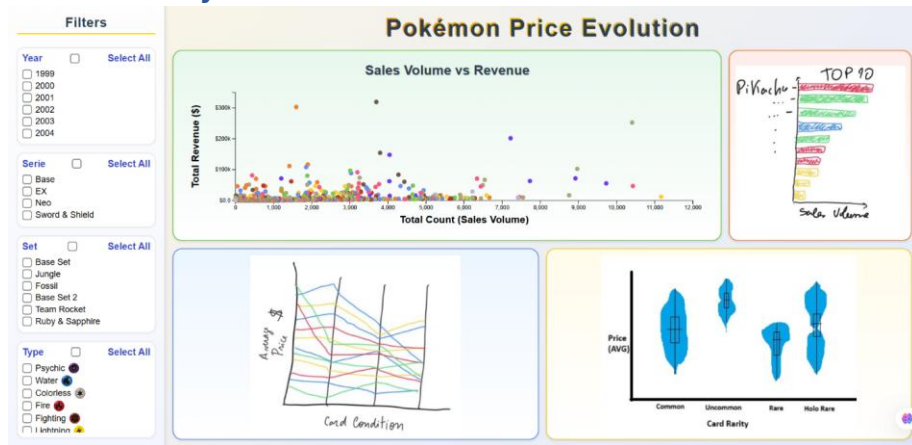
- **script.js** acts as the controller: it loads the dataset, initializes filters, applies user selections, and calls each chart module. It also manages global integration by storing and propagating the selected card across views.
- **Chart modules (Idioms folder)**: each chart has its own dedicated file (scatterplot.js, barchart.js, line chart.js, ViolinPlot.js). Each file exports a createX function that is responsible for:
 - Preprocessing the filtered data for its idiom
 - Drawing the visualization
 - Handling idiom-specific interactions
- **type_colors.js** centralizes color definitions for card types, ensuring a consistent visual language across idioms.

4. Data Flow

The prototype follows a clear data pipeline:



Dashboard Layout



Commented [EC1]: Put updated version of the dashboard
(substitute charts with images?)

The layout is structured around a consistent left-hand filter panel and a set of coordinated visualizations that update dynamically based on user selections. The filter panel provides stacked checkboxes for year, series, set, and type, enabling users to refine the dataset according to multiple criteria. In cases where no filters are applied, the dashboard displays the complete dataset, whereas incompatible filter selections result in no visualizations being rendered.

The central visualization space is organized into four main sections. In the upper-left, a scatter plot depicts the relationship between sales volume and total revenue, allowing for the identification of trends and outliers within the dataset. Positioned to the upper-right, a horizontal bar chart highlights the top ten most popular Pokémon based on sales volume, facilitating straightforward comparative analysis between individual cards. The lower-left section presents changes in average price across card conditions, conceptualized as a line chart in the sketch and implemented through multi-line plots in the prototype. Finally, the lower-right section features a Violin Plot illustrating price distributions across different rarity categories, thereby offering insights into market variability and value concentration.

Overall, the prototype demonstrates a faithful implementation of the sketch while improving clarity, interaction, and analytical value. The transition from conceptual layout to interactive prototype ensured that the dashboard retained its intended structure while providing a practical and informative tool for data exploration.

Data Processing

The Idiom we decided to start this first prototype with was the Scatterplot that answers tasks 4 and 5 respectively, Does type influence the popularity and price of a card by set and by year? and Pokémon Card Sales Volume vs. Revenue per Year, Type, Rarity and Name?. For the scatterplot idiom, the raw dataset could not be used directly. The original dataset contains multiple rows per card, each with attributes such as the card name, release year, set name, series, card(Pokémon) type, the number of cards sold(count), and the average sale price(avg). This data required preprocessing before it could be visualized effectively.

1. Filtering by user-selected criteria

Before aggregation, the dataset is filtered according to the user's choices in the sidebar filters (year, series, set, and type). This ensures that only the relevant subset of the dataset is passed to the scatterplot function.

2. Grouping the data

Since multiple rows can correspond to the same card (same name, series, and set), the data must be aggregated. Using `d3.rollup`, the prototype defines each individual card as (Card name, Series name and Set name). This allows the visualization to display each unique card as a single point, rather than duplicating it across multiple rows or hiding lots of data, since there are many Pokémon with the same name across sets and series.

3. Aggregating numerical values

For each group, the following metrics are computed:

- **Total Count (Sales Volume):** the sum of all count values in the group. This is used as the x-axis position.
- **Total Revenue:** the sum of (count * avg) for all rows in the group. This is used as the y-axis position.
- **Card Type:** since the type is categorical, the type of the first row in the group (type of the Pokémon) is kept. This determines the point's color.

4. Ranking

After aggregation, cards are sorted by their total count. Each card is assigned a **popularity rank** (1 = most sold). This rank is later displayed in the tooltip to provide additional context to the user.

5. Scaling to the visualization space

Once aggregated, the data is mapped to the scatterplot axes:

- The **x-axis** scale is based on the maximum sales volume across all cards.
- The **y-axis** scale is based on the maximum revenue.

Both scales are defined with `d3.scaleLinear` and expanded with `.nice()` for readability.

6. Preparing for interactivity

Each data point is augmented with information needed for interactive linking: name, series, set, type, sales volume, revenue, and rank. This ensures that when a point is clicked, the corresponding data can be shared across the other visualizations (bar chart, line chart, etc.).

Chart Interaction

The scatterplot provides several interactive features designed to help users explore the data more effectively and to support integration with the other visual idioms in the dashboard.

1. Hover (Tooltips)

When the user hovers over a data point (circle), a tooltip appears near the cursor. This tooltip provides detailed information about the selected card, including:

- Card name
- Series and set
- Type(s) with corresponding icons
- Total sales volume
- Total revenue
- Popularity rank

The border and header of the tooltip are styled according to the card's type color, reinforcing the visual mapping. This immediate feedback reduces the cognitive effort required to connect a point to its attributes.

3. Click-to-Select (Linked Highlighting Across Views)

Clicking a point toggles its selection. When a card is selected:

- The selected circle remains fully opaque while all other points fade (reduced opacity).
- The selection is **propagated to other charts** (bar chart, line chart, Violin Plot). The same card is highlighted in those views, creating a **linked-view system** where exploration in one chart informs the others.
- Clicking the same point again clears the selection, restoring the full dataset view.

4. Global Deselect

Clicking on empty space (anywhere outside points or controls) resets the selection. This ensures users can easily return to the full, unfiltered view without reloading or changing filters.

5. Cursor Feedback

Circles change the cursor to a pointer on hover, signaling interactivity and encouraging exploration.

6. Integration with Filters

The scatterplot dynamically updates when the user applies filters (year, series, set, type). Points outside the filter are removed, and scales are recalculated. This maintains consistency across the dashboard and ensures the user is always interacting with relevant data.

Chart Integration

The scatterplot is not an isolated view; it is fully integrated with the other idioms in the dashboard. The integration design follows the principle of **Linked Views**, where a user's interaction in one chart is reflected across all others.

1. Shared Data Model

All visualizations are built from the same filtered dataset. The filtering stage (years, series, sets, types) is executed only once, and the resulting subset is passed to each chart module. This ensures consistency across all idioms, since every view is derived from the same selection of data.

2. Selection Propagation

When a user clicks on a point in the scatterplot, the corresponding card is stored in a global variable. This object contains the card's key attributes (name, series, set), which serve as a unique identifier across all charts. This mechanism is **bidirectional**: selecting an element in any other chart updates the scatterplot as well, maintaining coherence.

3. Global Deselect

If the user clicks outside chart elements, the selected card resets and all views then restore their original appearance, reinforcing the sense of a unified dashboard state.

4. Extensibility

The integration is implemented in a modular way:

- Each chart module (scatterplot, bar chart, line chart, Violin Plot) has its own createX function (for example: createScatterplot(data: any, selector: any)).
- A central update function (updateSelectionAcrossPlots) handles the communication of selection states.
- Since the integration relies on a shared identifier (selectedCard), additional idioms can easily be added. As long as new charts listen to the same global selection variable, they will automatically integrate into the coordinated highlighting system.

5. Justification

Opacity was chosen as the main visual encoding for selection integration. This choice aligns with human perceptual strengths: opacity provides a smooth visual cue for de-emphasis without altering the fundamental encoding (position, color). Users can still see the full dataset for context while focusing on the selected card across views.